

UNIT-5

Shading and Hidden Surfaces

Session 1: Introduction to Illumination Model, Light Sources

Session Plan

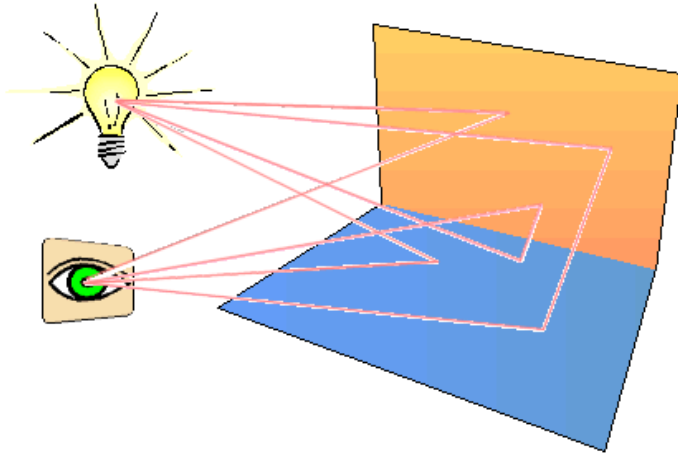
Time (in min)	Content	Learning Aid/Methodology	Faculty Approach	Typical Student Activity	Skill /Competency Developed	Course Outcome	Program Outcome
05	Introduction	Presentation and Quiz	Introduces Facilitates	Listens Participate Discuss	Knowledge Intrapersonal		
15	Illumination model	Presentation and Quiz	Introduces Facilitates	Listens Discuss	Knowledge Intrapersonal		
20	Light sources	Presentation Brain storming	Explain	Listen Understand Use	Knowledge Detect Application		
15	Examples of light sources	Brain storming Quiz	Explain Facilitates Summarizes	Listens Participate Discuss	Comprehension Application Interpersonal		
5	Conclusion and Summary	Key Words	List Facilitates	Identifies	Knowledge Comprehension		

Introduction

In order to show realistic picture on screen, we must color the object with specified color. For this we need to decide color of every object. Light from a light source falls on the object and get reflected back to the viewpoint (eye).

Lighting and Shading

- **Illumination:** The transport of light from a source to a point via direct and indirect paths.
- **Lighting:** The process of computing the luminous intensity reflected from a specified 3-D point
- **Shading:** The process of assigning a colors to a pixel



Illumination Models

Color is an important property of light. Lighting effects are described with models that consider the interaction with object surfaces. Lighting effects include light reflections, transparency, surface texture & shadows. A model for the interaction of light with a surface is called an illumination model. Physical illumination model involves the factors:- object type, object position relative to light sources & other objects & the light source condition that we set for a scene. Objects can be opaque, transparent, shiny, dull.

Diffuse Illumination

Rough, grainy surface scatters the reflected light in all directions. This scattered light is called diffused reflection. Hence the surface appears equally bright from all viewing directions. What we call the color of an object is the color of diffused reflection of the incident light. The part of incoming light will be absorbed & part of it is reflected by the surface. A blue object illuminated by a white light source, reflects the blue component of the white light source.

Surface color is also a factor for reflection. Dark surface reflects less light & light surface reflects more. The fractional amount of the incident light that is diffusely reflected can be set for each surface with parameter k_d , the **diffuse-reflection coefficient** or **diffuse reflectivity**.

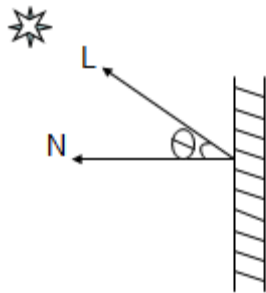
Value of reflectivity (k_d) is assigned in the range of 0 to 1 depending on the nature of surface (Light or Dark).

If a surface is exposed only to ambient light, hence the intensity of the diffuse reflection at any point

$$I = K_d * I_a \text{-----Eqn 1}$$

Point Source Illumination

Light from point source differs from ambient light as light comes from a particular direction & fixed place.



The position & orientation of the objects surface relative to the light source will determine how much light the surface will receive.

Near surface receive more light .

The illumination is decreased by a factor of $\cos\theta$, Where θ is the angle between the direction of light (L) & the direction Normal (N) to the plane. This angle is angle of incidence.

The amount of illumination depends on $\cos\theta$. If I_p is the intensity of the point source light then the diffuse reflection will be

$$I = K_d * I_p * \cos\theta \text{-----Eqn 2}$$

Place the value of $\cos\theta = N * L$ in Eqn 2

$$I = K_d * I_p * (N * L) \text{-----Eqn 3}$$

Now we can combine the Eqn 1 & Eqn 3 of ambient & point source to obtain expression for total diffuse reflection.

$$I = K_a * I_a + K_d * I_p * (N * L) \text{-----Eqn 4}$$

Where both K_a & K_d both will depend on surface material & will be in the range 0 to 1.

As the angle between direction of light & direction normal I.e. incidence angle increases, the incident light which falls on the surface will be less.



Common Types of Light Sources

- **Ambient light:** no identifiable source or direction

Intensity is the same at all points

This light does not have a direction (or .. it is the same in all directions)

Point Source

Given by a point p_0

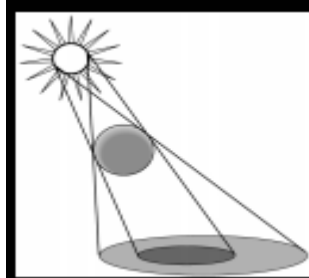
Light emitted from that point equally in all directions

Intensity decreases with square of distance

- **One Limitation of Point Sources**

Shading and shadows inaccurate

Example: penumbra (partial “soft” shadow)



- **Distant light:** given only by direction

Given by a vector v

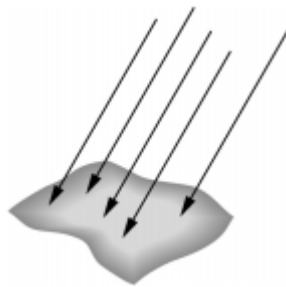
Intensity does not vary with distance (all distances are the same .. infinite!)

Light rays emanate in one direction from one point source – Parameters: direction

Attenuation proportional to cosine of angle between normal and light direction

No attenuation based on distance from source

Eg. Sun



- **Spotlight:** from source in direction – Cut-off angle defines a cone of light – Attenuation function (brighter in center)

Most complex light source in OpenGL

Light still emanates from point

Cut-off by cone determined by angle θ

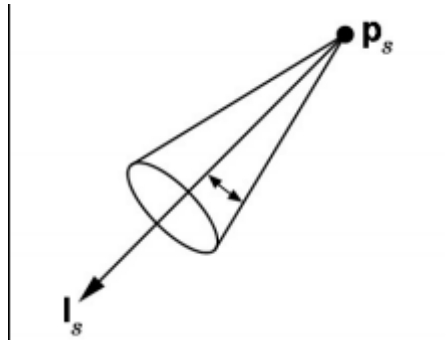
Light rays emanate in all directions from area source – Large circle such as spot lights –

Parameters: position or distance, cone angle, focus

Intensity attenuates in proportion to distance from source

Fluorescent tube – Slim rectangle, or other shapes.

Need to consider area of light source



- Light source described by a luminance – Each color is described separately – $I = [I_r \ I_g \ I_b]$
- T (I for intensity) – Sometimes calculate generically (applies to r, g, b)

Session 2: Ambient Light, Diffuse reflection

Session Plan

Time (in min)	Content	Learning Aid/Methodolog y	Faculty Approach	Typical Student Activity	Skill /Competency Developed	Course Outcom e	Prog ram Out com e
05	Introduction	Presentation and Quiz	Introduces Facilitates	Listens Participate Discuss	Knowledge Intrapersonal		
15	Ambient light	Presentation and Quiz	Introduces Facilitates	Listens Discuss	Knowledge Intrapersonal		
20	Diffuse reflection	Presentation Brain storming	Explain	Listen Understand Use	Knowledge Detect Application		
15	Diffuse refection in various aplication	Brain storming Quiz	Explain Facilitates Summarizes	Listens Participate Discuss	Comprehension Application Interpersonal		
5	Conclusion and Summary	Key Words	List Facilitates	Identifies	Knowledge Comprehension		

Ambient Light

- Ambient light means the light that is already present in a scene, before any additional lighting is added. It usually refers to natural light, either outdoors or coming through windows etc. It can also mean artificial lights such as normal room lights.
- A surface that is not exposed directly to a light source still will be visible if nearby objects are illuminated. In our basic illumination model, we can set a general level of brightness for a scene. This is a simple way to model the combination of light reflections from various surfaces to produce a uniform illumination called the ambient light, or background light.
- Non-directional light source
- Simulates light that has been reflected so many times from so many surfaces it appears to come equally from all directions
- intensity is constant over polygon's surface
- intensity is not affected by anything else in the world
- intensity is not affected by the position or orientation of the polygon in the world
- position of viewer is not important
- No 3-D information provided.
 - Generally used as a base lighting in combination with other lights.

$$I = I_a K_a$$

I: intensity

I_a : intensity of Ambient light

K_a : object's ambient reflection coefficient, 0.0 - 1.0 for each of R, G, and B

Diffuse Reflection

Diffuse Reflection

Diffuse reflection is uniform reflection of light with no directional dependence for the viewer, e.g., a matte surface such as cardboard. Diffuse reflection originates from a combination of internal scattering of light, i.e. the light is absorbed and then re-emitted, and external scattering from the rough surface of the object. An illumination model must handle both direct diffuse reflection, i.e., light coming directly from a source to a surface and then reflected to the viewer, and indirect diffuse reflection (or diffuse interreflections), that is light coming from a source, being reflected to a surface, then reflected to another surface, etc, and finally to the viewer.

Some local illumination models, such as those that are usually used in many [scan-line](#) rendering and [ray tracing](#) systems, handle the diffuse interreflection contribution by simply assuming it has

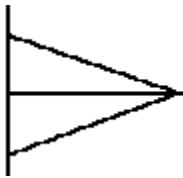
uniform intensity in all directions. This constant term is called the "ambient" contribution and it is considered a non-directional background light. [Radiosity](#) explicitly computes the diffuse interreflection term.

Diffuse reflection of ambient light

If we assume a uniform intensity of ambient light, I_a , then the intensity of the diffuse reflection at any point on the surface: $I = k_a * I_a$, where k_a = the ambient diffuse coefficient of reflection with: $0.0 \leq k_a \leq 1.0$. For colored surfaces there are three k_a values, one for red, green, and blue. We are approximating the ambient diffuse part of the reflection intensity function $R(j,q,l)$ by the constant k_a . For a dull, non-reflecting surface k_a is close to 0.0 and for a highly reflective surface, such as a mirror, k_a is close to 1.0.

Diffuse reflection of point light sources

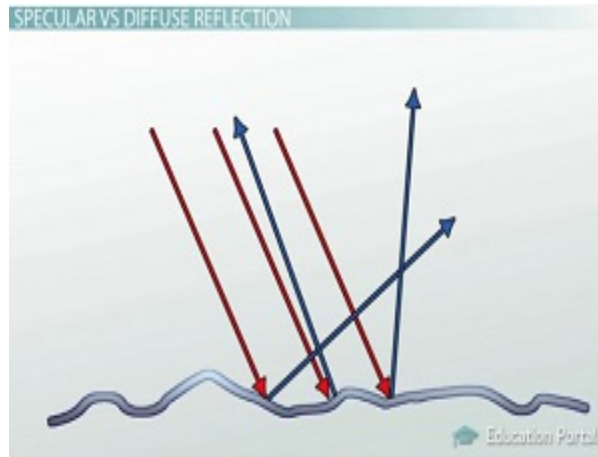
Consider a point source and assume the source is far from the surface so that incident light rays are parallel. Note: if the light source is near the surface then the light rays are not parallel as below:



Diffuse Reflection in Sound Waves

People who design auditoriums and recording studios are big fans of diffuse reflection. Imagine you are sitting in an auditorium, listening to someone play a trumpet. If the auditorium walls are smooth, the sound waves from the trumpet will bounce off the walls with specular reflection. If you're sitting in the right spot, you'll get a blast of intense sound waves as the reflected rays all hit you at once. If you move a few rows back, you might perceive the trumpet as sounding very soft. Specular reflection in an auditorium is not a good thing because different people hear different things depending on where they sit.

In order to break up the focus of the sound, people often use **acoustic diffusers**. These are wall and ceiling treatments that spread out sound waves by causing diffuse reflection. By engineering different shapes and angles into the walls, we can break up what would normally be specular reflections. Spreading out the sound waves results in a fuller, more uniform sound over the entire acoustic space. You'll often see diffusers inside recording studios and church sanctuaries. This is because diffuse reflection of sound waves sounds much better.



How reflective rays travel during diffuse reflection

Session 3: Specular Reflection, and the Phong model

Session Plan

Time (in min)	Content	Learning Aid/Methodology	Faculty Approach	Typical Student Activity	Skill /Competency Developed	Course Outcome	Program Outcome
05	Introduction	Presentation and Quiz	Introduces Facilitates	Listens Participate Discuss	Knowledge Intrapersonal		
15	Specular reflection	Presentation and Quiz	Introduces Facilitates	Listens Discuss	Knowledge Intrapersonal		
20	Phong model	Presentation Brain storming	Explain	Listen Understand Use	Knowledge Detect Application		
15	Difference between diffude and specular reflection	Brain storming Quiz	Explain Facilitates Summarizes	Listens Participate Discuss	Comprehension Application Interpersonal		
5	Conclusion and Summary	Key Words	List Facilitates	Identifies	Knowledge Comprehension		

Specular Reflection

Specular reflection can be observed on any shiny surface. The bright white highlight on a snooker ball is caused by specular reflection. Here, the surface acts as a mirror and reflects the incident ray. In specular reflection the photon does not interact with the surface at all and so the colour of the reflected light is the original colour of the incident light. Specular reflection can be seen on the sphere in the following picture:

- reflection off of shiny surfaces - you see a highlight
- shiny metal or plastic has high specular component
- chalk or carpet has very low specular component
- position of the viewer IS important in specular reflection

$$I = I_p \cos^n(\alpha) W(\theta)$$

I: intensity

I_p : intensity of point light

n: specular-reflection exponent (higher is sharper falloff)

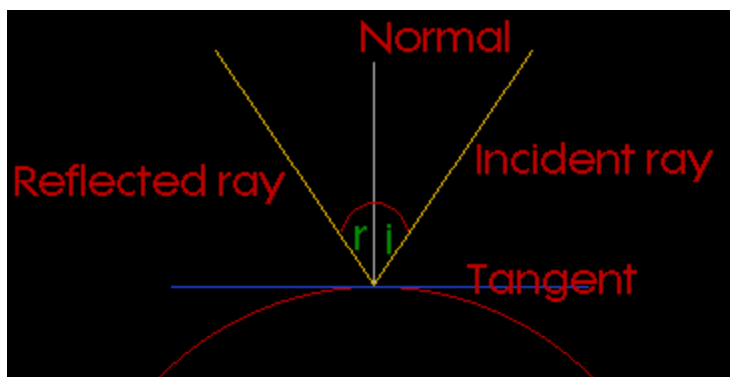
W: gives specular component of non-specular materials

So if we put all the lighting models depending on light together we add up their various components to get:

$$I = I_a K_a + I_p K_d(N \cdot L') + I_p \cos^n(\alpha) W(\theta)$$

Perfect Specular Reflection

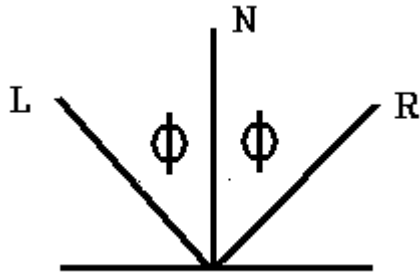
Here, the surface acts as a perfect mirror and reflects the incident ray without any attenuation.



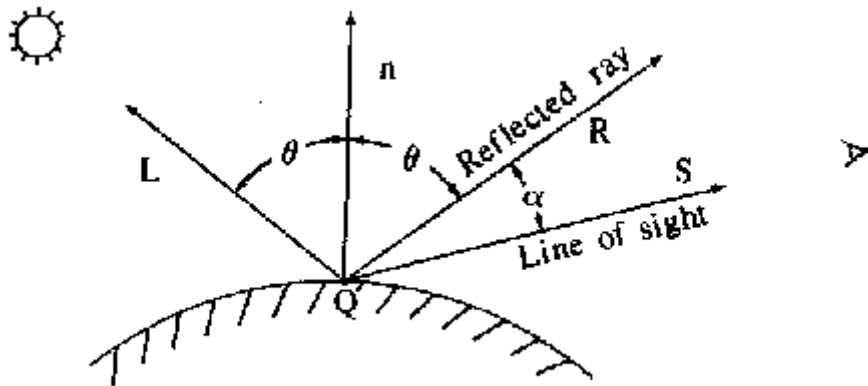
In the diagram, we have a sphere being struck by an incident ray at an angle i from the normal. Assuming the sphere is a perfect shiny surface, the resultant reflected ray will be emitted at the same angle from the other side of the normal (ie, in the diagram $r=i$).

PHONG MODEL

Specular reflection is when the reflection is stronger in one viewing direction, i.e., there is a bright spot, called a specular highlight. This is readily apparent on shiny surfaces. For an ideal reflector, such as a mirror, the angle of incidence equals the angle of specular reflection, as shown below.

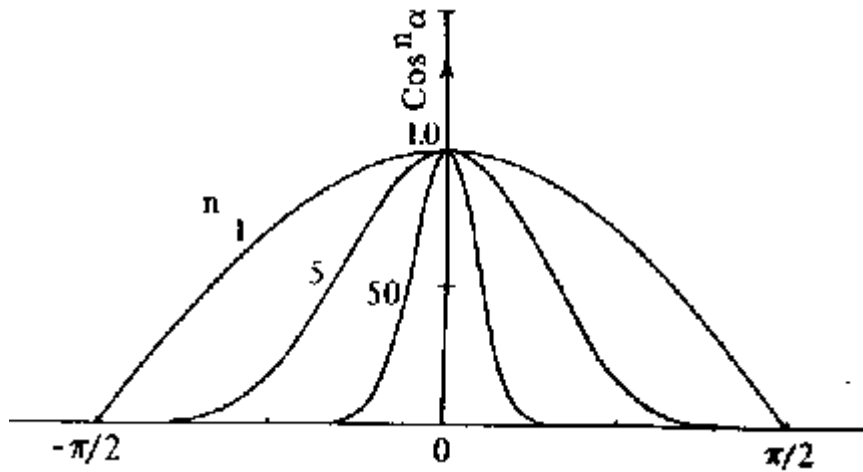


So if **R** is the direction of specular reflection and **V** is the direction of the viewer (located at the View Reference Point or **VRP**), then for an ideal reflector the specular reflection is visible only when **V** and **R** coincide. For real objects (not perfect reflectors) the specular reflectance can be seen even if **V** and **R** don't coincide, i.e., it is visible over a range of values (or a cone of values). The shinier the surface, the smaller the range for specular visibility. So a specular reflectance model must have maximum intensity at **R**, with an intensity which decreases as $f(a)$.

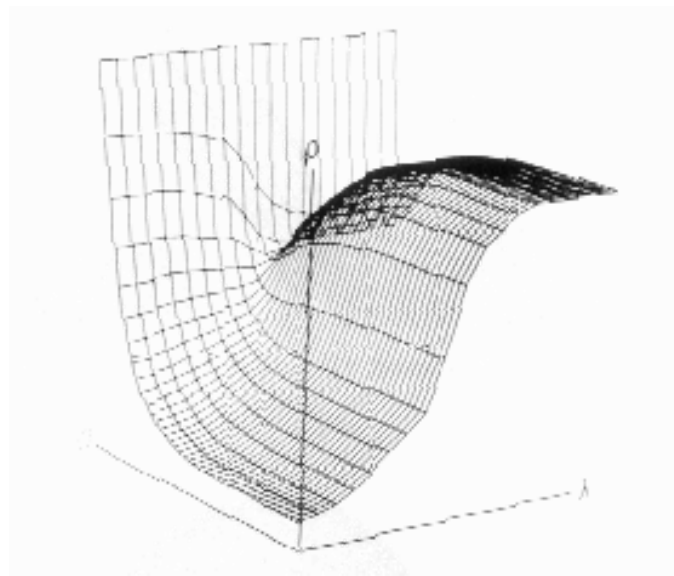


Phong Model (by Phong Bui-Tuong)

This is an empirical model, which is not based on physics, but physical observation. Phong observed that for very shiny surfaces the specular highlight was small and the intensity fell off rapidly, while for duller surfaces it was larger and fell off more slowly. He decided to let the reflected intensity be a function of $(\cos \alpha)^n$ with $n \geq 200$ for a shiny surface and n small for a dull surface. For a perfect reflector n equals infinity, and for a piece of cardboard n equals 0 or 1. In the diagram below we can see how the function $(\cos \alpha)^n$ behaves for different values of n .



Specular reflection is also a function of the light incidence angle q . An example is glass which has almost no specular reflectance for $q = 0$ degrees but a very high specular reflectance for $q > 80$ degrees. Some substances, such as copper, actually change color with change in the incidence angle, as shown in the following plot of the reflectance curve as a function of the incident angle for copper. .



Session 4: Combined diffuse and specular reflections with multiple light sources**Session Plan**

Time (in min)	Content	Learning Aid/Methodolog y	Faculty Approach	Typical Student Activity	Skill /Competency Developed	Course Outcom e	Prog ram Out com e
05	Introduction	Presentation and Quiz	Introduces Facilitates	Listens Participate Discuss	Knowledge Intrapersonal		
15	Diffuse reflection	Presentation and Quiz	Introduces Facilitates	Listens Discuss	Knowledge Intrapersonal		
20	specular reflections	Presentation Brain storming	Explain	Listen Understand Use	Knowledge Detect Application		
15	Combined diffuse and specular reflections with multiple light sources	Brain storming Quiz	Explain Facilitates Summarizes	Listens Participate Discuss	Comprehension Application Interpersonal		
5	Conclusion and Summary	Key Words	List Facilitates	Identifies	Knowledge Comprehension		

Combined Diffuse and Specular Reflections with Multiple Light Sources

For a single point light source, we can model the combined diffuse and specular reflections from a point on an illuminated surface as

$$\begin{aligned} I &= I_{\text{diff}} + I_{\text{spec}} \\ &= k_a I_a + k_d I_l (\mathbf{N} \cdot \mathbf{L}) + k_s I_l (\mathbf{N} \cdot \mathbf{H})^{n_s} \end{aligned} \quad (14-9)$$

Figure 14-19 illustrates surface lighting effects produced by the various terms in Eq. 14-9. If we place more than one point source in a scene, we obtain the light reflection at any surface point by summing the contributions from the individual sources:

$$I = k_a I_a + \sum_{i=1}^n I_{li} [k_d (\mathbf{N} \cdot \mathbf{L}_i) + k_s (\mathbf{N} \cdot \mathbf{H}_i)^{n_s}] \quad (14-10)$$

To ensure that any pixel intensity does not exceed the maximum allowable value, we can apply some type of normalization procedure. A simple approach is to set a maximum magnitude for each term in the intensity equation. If any calculated term exceeds the maximum, we simply set it to the maximum value. Another way to compensate for intensity overflow is to normalize the individual terms by dividing each by the magnitude of the largest term. A more complicated procedure is first to calculate all pixel intensities for the scene, then the calculated intensities are scaled onto the allowable intensity range.

Session 5: warn model, shading Algorithms Halftone,

Session Plan

Time (in min)	Content	Learning Aid/Methodology	Faculty Approach	Typical Student Activity	Skill /Competency Developed	Course Outcome	Program Outcome
05	Introduction	Presentation and Quiz	Introduces Facilitates	Listens Participate Discuss	Knowledge Intrapersonal		
15	warn model	Presentation and Quiz	Introduces Facilitates	Listens Discuss	Knowledge Intrapersonal		
20	shading Algorithms	Presentation Brain storming	Explain	Listen Understand Use	Knowledge Detect Application		
15	Halftone algorithm	Brain storming Quiz	Explain Facilitates Summarizes	Listens Participate Discuss	Comprehension Application Interpersonal		
5	Conclusion and Summary	Key Words	List Facilitates	Identifies	Knowledge Comprehension		

WARN MODEL

Warn Model

So far we have considered only point **light sources**. The Warn model provides a method for simulating studio lighting effects by controlling **light** intensity in different directions.

Light sources are modeled as points on a reflecting surface, using the Phong model for the surface points. Then the intensity in different directions is controlled by selecting values for the Phong exponent. In addition, **light** controls, such as “barn doors” **and** spotlighting, used by studio photographers can be simulated in the Warn model. *Flaps* are used to control the amount of **light** emitted by a source in various directions. Two flaps are provided for each of the *x*, *y*, **and** *z* directions. *Spotlights* are used to control the amount of **light** emitted within a cone **with** apex at a point-source position. The Warn model is implemented in

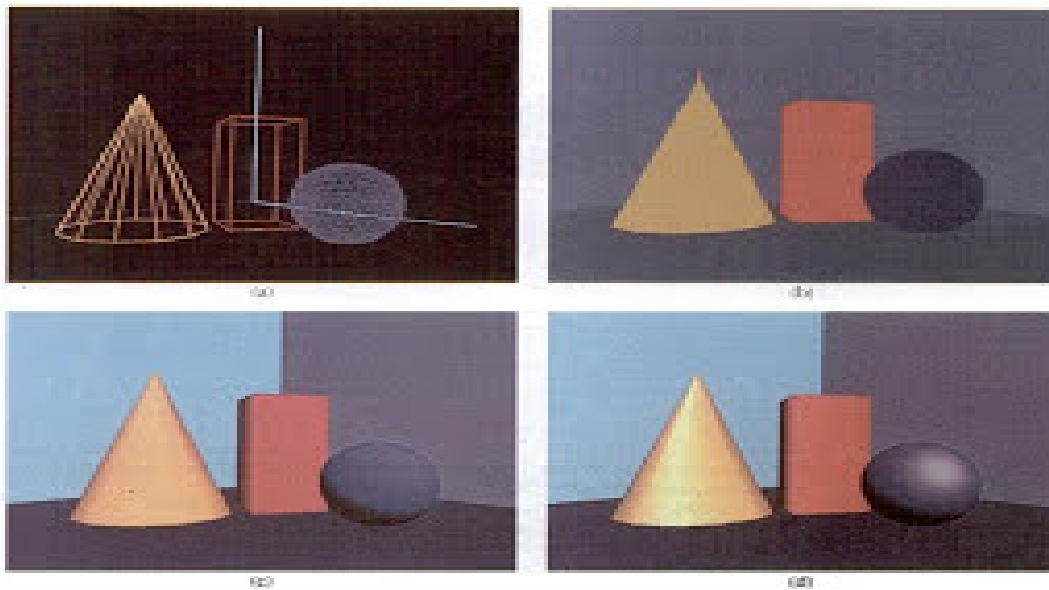


Figure 14-19

A wireframe scene (a) is displayed only with ambient lighting in (b), and the surface of each object is assigned a different color. Using ambient light and diffuse reflections due to a single source with $R_s = 0$ for all surfaces, we obtain the lighting effects shown in (c). Using ambient light and both diffuse and specular reflections due to a single light source, we obtain the lighting effects shown in (d).

PHONG-, and Fig. 14-20 illustrates lighting effects that can be produced with this model.

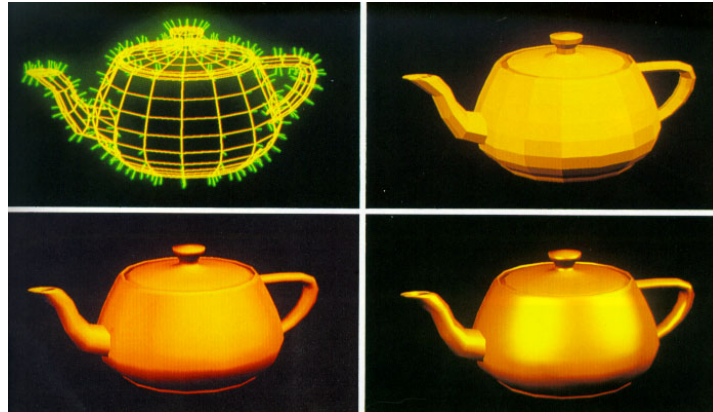
Incremental Shading Techniques

Most images currently generated in computer graphics apply the Phong reflection model to polygon mesh models. Polygon mesh models are distinguished by the fact that the geometric information is only available at the vertices of a polygon. Incremental or interpolative shading techniques apply simple reflection models to polygons, calculating intensities at the vertices, and then interpolating values for interior points. Interior points are evaluated in scan line order and the schemes are easily integrated into a Z-buffer or a scan line hidden-surface removal algorithm.

The term 'shading' is loosely used to signify the application of a 'point' reflection model over the entire surface of an object. The term 'rendering' appears to be used to mean the complete process of going from an object database to the final shaded object on a screen. This section deals only with the problem of applying simple reflection models to polygon mesh objects. The next section looks at how these methods are integrated with scan conversion and hidden surface removal to form a complete rendering system.

Two incremental shading techniques are common; Gouraud interpolation (1971) and Phong interpolation (1975). Phong interpolation gives more accurate highlights and is generally the preferred model. The Gouraud method tends to be confined to applications where the diffuse component is sufficient, or to preview images prior to final rendering.

Phong shading is more expensive than Gouraud shading and obeys the first law of computer graphics, which appears to be that the required processing time grows exponentially with perceived image quality. However, Gouraud shading has become almost standard in recent graphics hardware and Phong shading is also available.



Shading examples: top left - lines with normals; top right - faceted; bottom left - Gouraud; bottom right – Phong

These notes deal with both these methods . Doing shading calculations efficiently is a neglected topic in computer graphics. Phong calculations can greatly exceed more than 50% of the total rendering time and addressing the problem of shading using an efficient method should be considered to be just as important as the quality of the reflection model. It is mandatory in areas such as three-dimensional animation where large numbers of frames have to be generated.

Another problem that must be addressed by interpolative shading techniques is final polygon visibility. Polygon boundaries should be invisible in the final shaded version and some impression of the original surface that the polygon mesh approximates restored. We can thus identify two functions of a shading scheme for polygon meshes:

- (1) to use some interpolative method for interior points, and
- (2) to diminish the visibility of the polygon mesh approximation.

The first shading schemes to be used in computer graphics were developed by Bouknight (1970) and Wylie et al. (1967). Wylie et al.'s method calculated an intensity at the vertices of triangular facets on the basis of distance from the viewpoint and then used linear interpolation to assign an intensity to interior points. In Bouknight's work the emphasis was on hidden surface removal and the polygons were 'constant' shaded. That is, a single intensity was calculated for each polygon and used over its entire area. This method, although it certainly imparts an impression of three dimensionality, leaves the polygon mesh structure glaringly obvious.

Session 6: Gouraud and Phong Shading

Session Plan

Time (in min)	Content	Learning Aid/Methodology	Faculty Approach	Typical Student Activity	Skill /Competency Developed	Course Outcome	Program Outcome
05	Introduction	Presentation and Quiz	Introduces Facilitates	Listens Participate Discuss	Knowledge Intrapersonal		
15	Gouraud shading algorithm	Presentation and Quiz	Introduces Facilitates	Listens Discuss	Knowledge Intrapersonal		
20	Phong shading algorithm	Presentation Brain storming	Explain	Listen Understand Use	Knowledge Detect Application		
15	Examples	Brain storming Quiz	Explain Facilitates Summarizes	Listens Participate Discuss	Comprehension Application Interpersonal		
5	Conclusion and Summary	Key Words	List Facilitates	Identifies	Knowledge Comprehension		

Gouraud Shading

The generally acknowledged first scheme that overcame the disadvantages of constant shading of polygons uses bilinear intensity interpolation. This is known as Gouraud shading (1971). It is a simple and economic scheme that does not entirely eliminate the visibility of polygons. It suffers from Mach banding - where piecewise linear intensity changes across a polygon boundary trigger the human visual system into perceiving the boundary as a bright band. The standard explanation for this is that the human visual system is sensitive to the second derivative of intensity because of our need to detect and enhance edges.

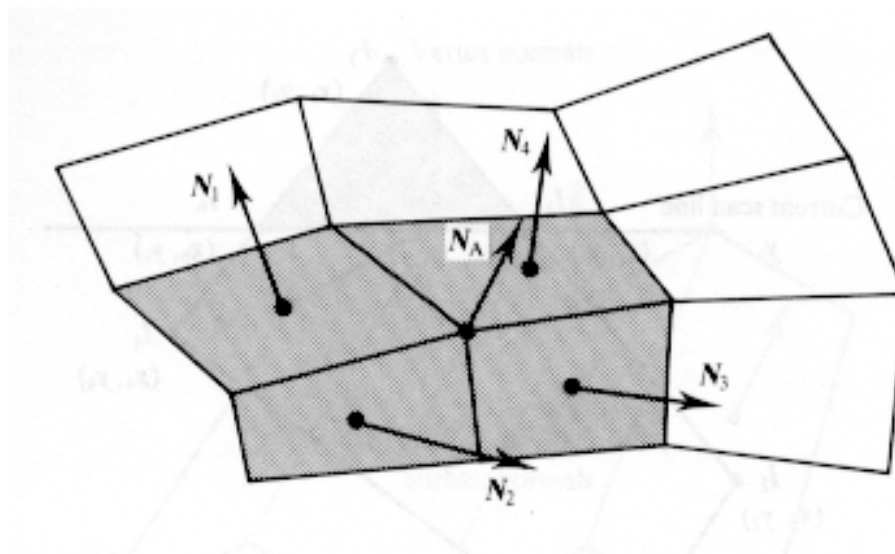
The Gouraud scheme is normally restricted to the diffuse component of the reflection model developed in Chapter 2. This is because the shape of the specular highlights, using this scheme, depends strongly on the underlying polygon mesh. An obvious highlight example that Gouraud shading misses altogether is the case of a highlight in the middle of a polygon. If there is no highlight component at any vertex then there is no way that the highlight will be recovered by the interpolation. The diffuse component as we have seen previously is expressed as:

$$I_i k_d (\mathbf{L} \cdot \mathbf{N}) / (r + k)$$

If we assume the light source is at infinity then $\mathbf{L} \cdot \mathbf{N}$ is constant over the surface of the polygon. The only variable is r , the distance of the view point. If we ignore the fact that the variation of r over the polygon is non-linear (or even ignore r altogether), then we can calculate intensities at the vertices and use bilinear interpolation to calculate all other intensities.

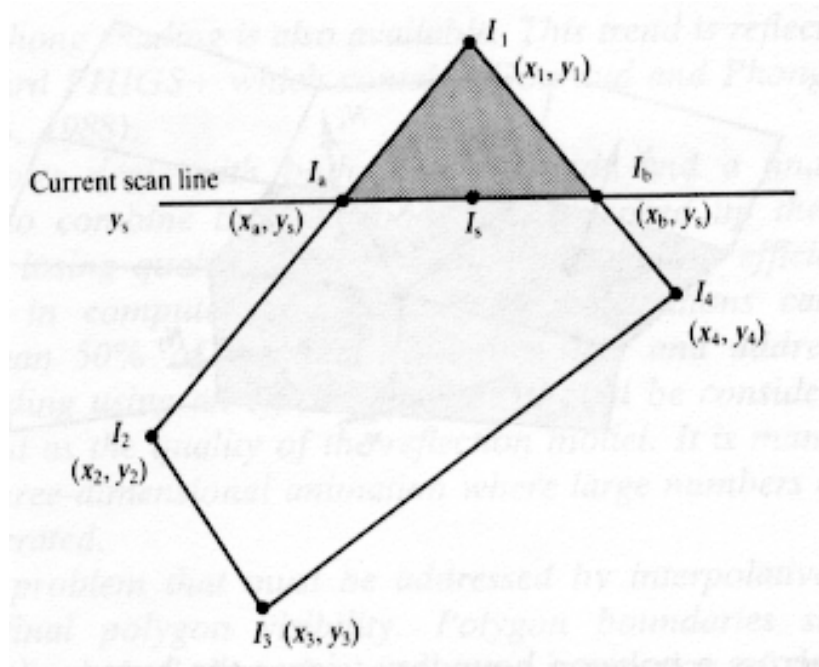
The technique first calculates the intensity at each vertex of the polygon. The normal $\mathbf{N}$ used in this calculation is the so-called vertex normal and this is calculated as the average of the normal of the polygons that share the vertex (see the figure below). This is an important feature of the method and the vertex normal is an approximation to the true normal of the surface (which the polygon mesh represents) at that point.

A pass through the data structure storing the object will calculate an intensity I_v for each vertex. The interpolation process that calculates the intensity over a polygonal surface can then be integrated with a scan conversion algorithm that evaluates the screen position of the edges of a polygon from the vertices in the data structure.



The vertex normal N_A is the average of the normals N_1 , N_2 , N_3 and N_4 , the normals of the polygons that meet at the vertex.

The intensities at the edge of each scan line are calculated from the vertex intensities and the intensities along a scan line from these (see figure below). The interpolation equations are as follows:



The notation used for the intensity interpolation equations.

For computational efficiency these equations are implemented as incremental calculations. This is particularly important in the case of the third equation, which is evaluated for every pixel. If we define Δx to be the incremental distance along a scan line then ΔI , the change in intensity, from one pixel to the next is

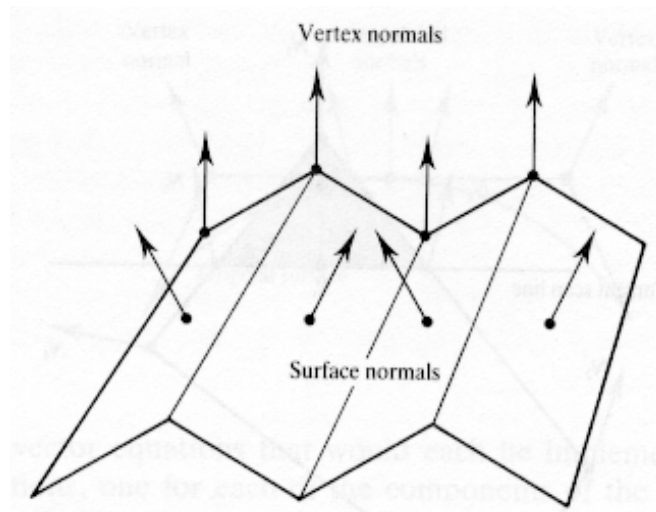
$$\Delta I_s = \frac{\Delta x}{x_b - x_a} (I_b - I_a)$$

$$I_{s,n} = I_{s,n-1} + \Delta I_s$$

Apart from Mach banding, two well-known errors that can be introduced in Gouraud shading are:

1) Anomalies can appear in animated sequences because the intensity interpolation is carried out in screen coordinates from vertex normals calculated in world coordinates. This is not invariant with respect to transformations such as rotation and in animated sequences frame-to-frame disturbances can be caused by this.

2) The process of averaging surface normals to provide vertex normals for the intensity calculation can cause errors which result in, for example, corrugations being smoothed out. The figure below shows three normals all pointing in the same direction. This would result in a visually flat surface. Although this can be a problem, it should not occur providing a sufficiently dense polygonal surface has been used in the model.

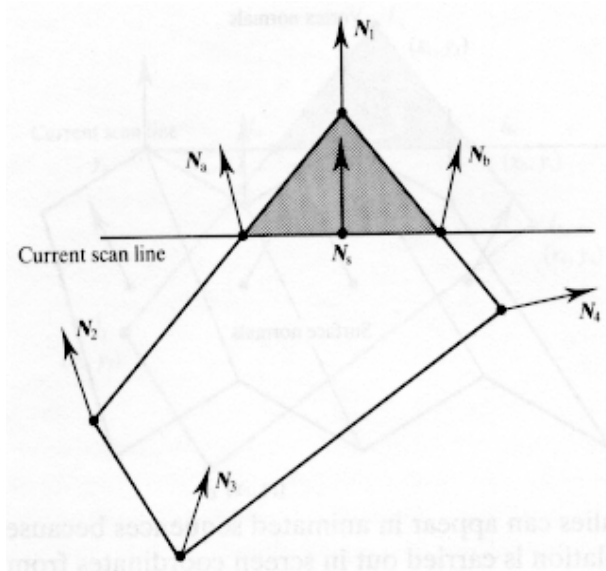


A regularly corrugated surface producing identical vertex normals from non-identical surface normals.

Phong interpolation

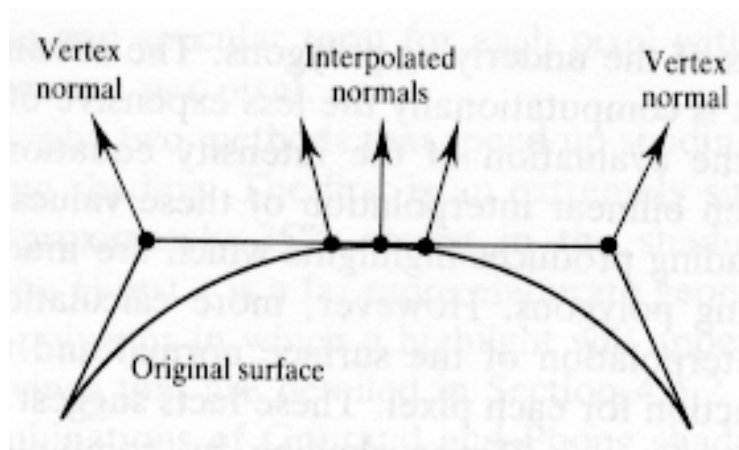
A method due to Bui-TuongPhong (1975) overcomes some of the disadvantages of Gouraud shading and specular reflection can be successfully incorporated in the scheme. In particular we can now have a specular highlight in the middle of a polygon despite the fact that each of the vertex normal angles would not produce a highlight. The features of the method are:

- 1) Bilinear interpolation is still used so that points interior to polygons can be calculated incrementally.
- 2) The attributes interpolated are the vertex normals, rather than vertex intensities. These are calculated, as before, by averaging the normal vectors of the surfaces that share the vertex.
- 3) A separate intensity is evaluated for each pixel from the interpolated normals.
- 4) Again we need to assume that both the light source and the viewpoint are at infinity so that the intensity at a point is a function only of the interpolated normal.



In the Phong method vector interpolation replaces intensity interpolation.

The first stage in the process is the same as for the Gouraud method - for any polygon we evaluate the vertex normals. For each scan line in the polygon we evaluate by linear interpolation the normal vectors at the end of each line (see the figure above). These two vectors N_a and N_b are then used to interpolate N_s . We thus derive a normal vector for each point or pixel on the polygon that is an approximation to the real normal on the curved surface approximated by the polygon. This feature accounts for the quality of Phong shading. N_s , the interpolated normal vector, is then used in the intensity calculation. This idea is represented in the figure below, which shows that vector interpolation tends to restore the curvature of the original surface that has been approximated by a polygon mesh. Referring to the notation used above we have:



Vector interpolation tends to 'restore' curvature

$$N_a = \frac{1}{y_1 - y_2} [N_1(y_s - y_2) + N_2(y_1 - y_s)]$$

$$N_b = \frac{1}{y_1 - y_4} [N_1(y_s - y_4) + N_4(y_1 - y_s)]$$

$$N_s = \frac{1}{x_b - x_a} [N_s(x_b - x_s) + N_b(x_s - x_a)]$$

These are vector equations that would each be implemented as a set of three equations, one for each of the components of the vectors in world space. This makes the interpolation phase three times as expensive as Gouraud shading. In addition there is an application of the Phong model intensity equation at every pixel. Incremental computations can be employed as with intensity interpolation, and for example, would be implemented as:

$$N_{sx,n} = N_{sx,n-1} + \Delta N_{sx}$$

$$N_{sy,n} = N_{sy,n-1} + \Delta N_{sy}$$

$$N_{sz,n} = N_{sz,n-1} + \Delta N_{sz}$$

where N_{sx} , N_{sy} and N_{sz} are the components of a general scanline normal vector N_s and

$$\Delta N_{sx} = \frac{\Delta x}{x_b - x_a} (N_{bx} - N_{ax})$$

$$\Delta N_{sy} = \frac{\Delta x}{x_b - x_a} (N_{by} - N_{ay})$$

$$\Delta N_{sz} = \frac{\Delta x}{x_b - x_a} (N_{bz} - N_{az})$$

Comparison of Gouraud and Phong shading

Gouraud shading is effective for shading surfaces which reflect light diffusely. Specular reflections can be modeled using Gouraud shading, but the shape of the specular highlight produced is dependent on the relative positions of the underlying polygons. The advantage of Gouraud shading is that it is computationally the less expensive of the two models, only requiring the evaluation of the intensity equation at the polygon vertices, and then bilinear interpolation of these values for each pixel.

Phong shading produces highlights which are much less dependent on the underlying polygons. However, more calculations are required, involving the interpolation of the surface normal and the evaluation of the intensity function for each pixel.

Session 7: introduction to Hidden surfaces, back-face removal algorithm

Session Plan

Time (in min)	Content	Learning Aid/Methodolog y	Faculty Approach	Typical Student Activity	Skill /Competency Developed	Course Outcom e	Prog ram Out com e
10	Introduction	Presentation and Quiz	Introduces Facilitates	Participate Discuss	Knowledge Intrapersonal		
10	Hidden surfaces and Hidden line	Presentation Brain storming Quiz	Explain Facilitates	Listens Participate Discuss	Knowledge Comprehension Application		
15	Back – face removal algorithm	Presentation Brain storming Quiz	Explain Facilitates	Listens Participate Discuss	Knowledge Finding Interpreting Use Application		
10	Techniques to solve full hidden-surface and hidden-line problems	Presentation Brain storming Quiz	Explain Facilitates	Listens Participate Discuss	Knowledge Identify Compare Implement Application		
10	Conclusion and Summary	Presentation Brain storming Quiz	Explain Facilitates	Listens Participate Discuss	Knowledge Naming Carrying out Application		
5	Introduction	Key Words	List Facilitates Summarizes	Identifies	Knowledge Comprehension Intrapersonal		

Hidden Surfaces

Opaque objects that are closer to the eye and in the line of sight of other objects will block those objects those objects from view. In fact, some surfaces of these opaque objects themselves are not Visible because they are eclipsed by the objects' visible parts. The surfaces that are blocked or hidden from view must be "removed" in order to construct a realistic view of the 3D scene

The identification and removal of these surfaces is called the hidden – surface problem. The solution involves the determination of the closest visible surface along each projection line. There are many different hidden –surface algorithms. Each can be characterized as either an image-space method, in which the pixel grid is used to guide the computational activities that determine visibility at the pixel level, or an object-space method, in which surface visibility is determined using continuous models in the object space (or its transformation) without involving pixel based operations.

Notice that the hidden – surface problem has ties to the calculation of shadows. If we place a light source, such as a bulb, at the viewpoint, all surfaces that are visible from the viewpoint are lit directly by the light source and all surfaces that are hidden from the viewpoint are in the shadow of some opaque objects blocking the light.

Depth Comparisons

We assume that all coordinates (x,y,z) are described in the normalized viewing co-ordinates system. Any hidden-surface algorithm must determine which edges and surfaces are visible either from the center of projection for perspective projections or along the direction of projection for parallel projections.

The question of visibility reduces to this : given two points $P_1 (x_1, y_1, z_1)$ and $P_2 (x_2, y_2, z_2)$, does either point obscure the other? This is answered in two steps:

- 1 Are P_1 and P_2 on the same projection line?
- 2 If not, neither point obscures the other. If so, a depth comparison tells us which point is in front of the other.

For an orthographic parallel projection onto the xy plane, P_1 and P_2 are on the same projector if $x_1 = x_2$ and $y_1 = y_2$. In this case, depth comparison reduces to comparing z_1 and z_2 . If $z_1 < z_2$, then P_1 obscures P_2 [see Fig. 1(a)].

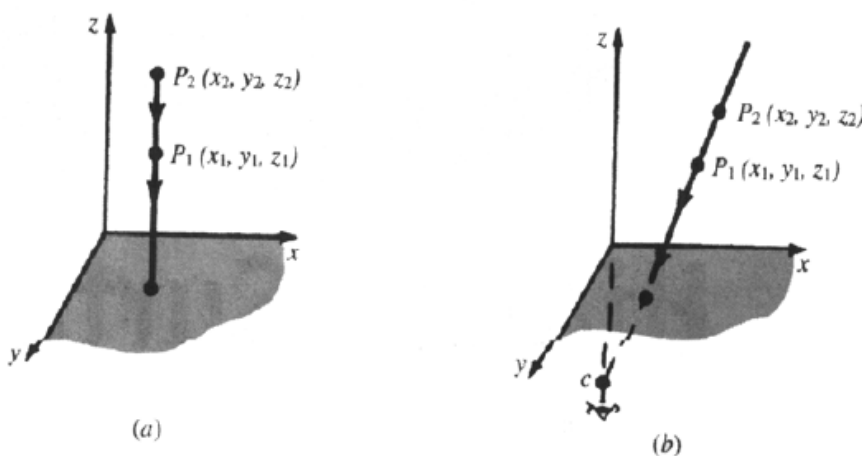


Fig. 1

For a perspective projection [see. Fig. 1(b)], the calculations are more complex.

However, this complication can be avoided by transforming all-three dimensional objects so that parallel projection of the transformed object is equivalent to a perspective projection of the original object (see Fig. 2). This is done with the use of the perspective to parallel transform T_p .

If the original object lies in the normalized perspective view volume, the normalized perspective to parallel transforms

$$NT_p = \begin{pmatrix} \frac{1}{2} & 0 & \frac{1}{2} & 0 \\ 0 & \frac{1}{2} & \frac{1}{2} & 0 \\ 0 & 0 & \frac{1}{1-z_f} & \frac{-z}{1-z_f} \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

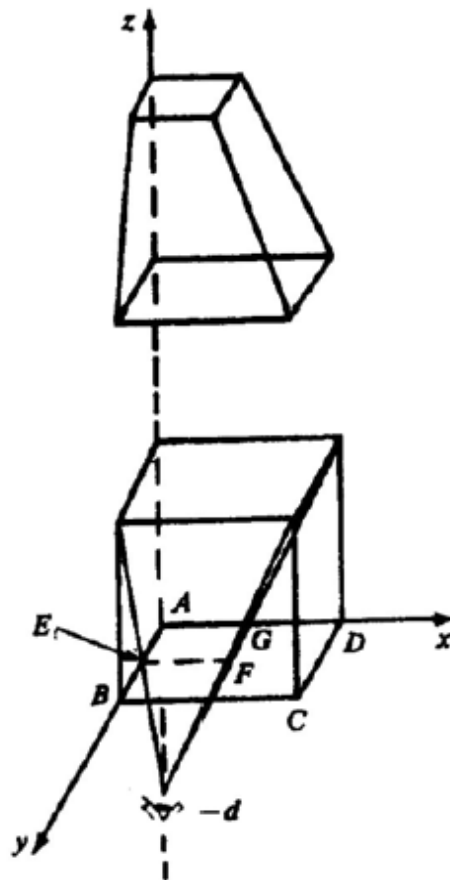


Fig 2

(where z_f is the location of the front clipping plane of the normalized perspective view volume) transforms the normalized perspective view volume into the unit cube bounded by $0 \leq x \leq 1$, $0 \leq y \leq 1$, $0 \leq z \leq 1$. We call this cube the normalized display space. A critical fact is that the normalized perspective to parallel transforms preserves lines, planes, and depth relationships.

If our display device has display coordinates $H \times V$, application of the scaling matrix

$$S_{H,V,1} = \begin{pmatrix} H & 0 & 0 & 0 \\ 0 & V & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

transforms the normalized display space $0 \leq x \leq 1$, $0 \leq y \leq 1$, $0 \leq z \leq 1$ onto the region $0 \leq x \leq H$, $0 \leq y \leq V$, $0 \leq z \leq 1$. We call this region the display space. The display transforms DT_p .

$$DT_p = S_{H,V,1} \cdot NT_p$$

transforms the normalized perspective view volume onto the display space.

Clipping must be done against the normalized perspective view volume prior to applying the transform NT_p . An alternative to this is to combine NT_p with normalizing transformation N_{per} , forming the single transformation

$$NT'_p = NT_p \cdot N_{per}.$$

Then clipping is done in homogeneous coordinate space.

We now describe several algorithms for removing hidden surfaces from scenes containing objects defined with planar (i.e., flat), polygonal faces. We assumed that the displays transform DT_p has been applied (if a perspective projection is being used), so that we always deal with parallel projections in display space.

Hidden Line/ Surface Elimination Methods

Although there are special-purpose hidden –line algorithms, each of the above algorithms can be modified to eliminate hidden lines or edges. This is especially useful for wire frame polygonal models where the polygons are unfilled. The idea is to use a color rule, which fills all the polygons with the background color – say the edges and lines a different color – say white. The use of a hidden surface algorithm now becomes a hidden –line algorithm.

Back-Face Removal

Object surfaces that are oriented away from the viewer are called back-faces. The back-faces of an opaque polyhedron are completely blocked by the polyhedron itself and hidden from view. We can therefore identify and remove these back-faces based solely on their orientation without further processing (projection and scan-conversion) and without regard to other surfaces and objects in the scene.

Let $N = (A, B, C)$ be the normal vector of a planar polygonal face, with N pointing in the direction the polygon is facing. Since the direction of viewing is the direction of the positive z axis (see Fig. 7.30), the polygon is facing away from the viewer when $C > 0$, (the angle between N and the z axis is less than 90°). The polygon is also classified as a back-face when $C = 0$ since in this case it is parallel to the line of sight and its projection is either hidden as a back-face when $C = 0$, since in this case it is parallel to the line of sight and its projection is either hidden or overlapped by the edge(s) of some visible polygon(s).

Although this method identifies or removes back-faces quickly it does not handle polygons that face the viewer but are hidden (partially or completely) behind other surfaces. It can be used as a preprocessing step for other algorithms.

Session 8: Algorithms: Depth buffer (z), Depth sorts (Painter),

Time	Content	Learning	Faculty	Typical	Skill	Course	Prog
------	---------	----------	---------	---------	-------	--------	------

(in min)		Aid/Methodology	Approach	Student Activity	/Competency Developed	Outcome	ram Outcome
10	Introduction	Presentation and Quiz	Introduces Facilitates	Participate Discuss	Knowledge Intrapersonal		
15	Shading algorithm	Presentation Brain storming	Explain Facilitates	Listens Participate Discuss	Identify Classify Carry out		
15	Depth Buffer(z)	Presentation Brain storming Quiz	Explain Facilitates	Listens Participate Discuss	Knowledge Comprehension Implement Application		
10	Depth sorts (Painter)	Presentation Brain storming Quiz	Explain Facilitates	Listens Participate Discuss	Identify Classify Execute Design		
10	Conclusion and Summary	Key Words	List Facilitates Summarizes	Identifies	Knowledge Comprehension Intrapersonal		

Z – Buffer Algorithm

We say that a point in display space is “seen” from pixel (x,y) if the projection of the point is scan converted to this pixel. The Z-buffer algorithm essentially keeps track of the smaller z coordinate (also called the depth value) of those point which are seen from pixel (x,y). These Z values are stored in what is called the Z buffer.

Let $Z_{buf}(x, y)$ denote the current depth value that is stored in the Z buffer at pixel (x, y). We work with the (already) projected polygons P of the scene to be rendered.

The Z-buffer algorithm consists of the following steps.

1. Initialize the screen to a background color. Initialize the Z buffer to the depth of the back clipping plane. That is, set

$$Z_{buf}(x, y) = Z_{back}, \text{ for every pixel } (x, y)$$

2. Scan-convert each (projected) polygon P in the scene and during this scan–conversion process, for each pixel (x, y) that lies inside the polygon.

- a. Calculate $Z(x,y)$ the depth of the polygon at pixel (x, y).

b. If $Z(x,y) < Z_{buf}(x,y)$, set $Z_{buf}(x,y) = Z(x,y)$, and set the pixel value at (x,y) to the color of the polygon P at (x,y) . In Fig. 3, points P_1 and P_2 are both scan-converted to pixel (x,y) ; however, since $z_1 < z_2$, P_1 will obscure P_2 and the P_1 z value z_1 , will be stored in the Z buffer.

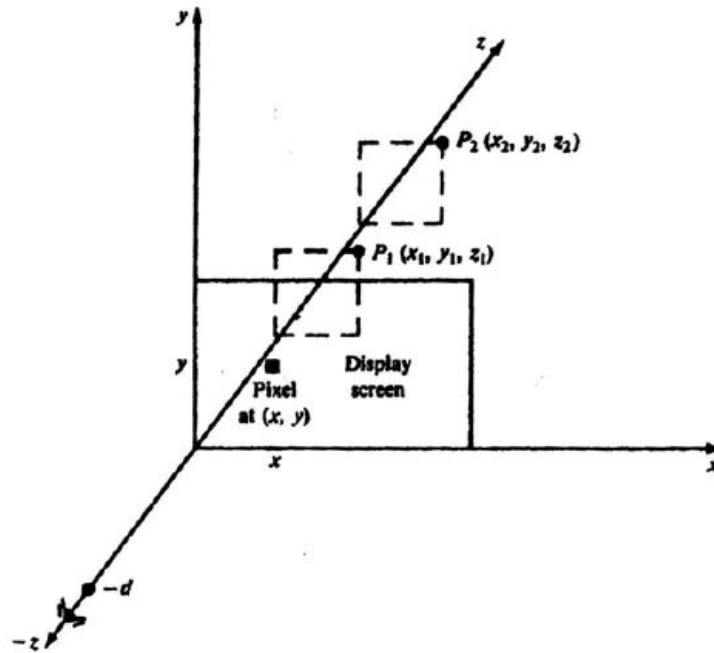


Fig. 3

Although the Z-buffer algorithm requires Z-buffer storage proportional to the number of pixels on the screen, it does not require additional memory for storing all the objects comprising the scene. In fact, since the algorithm process polygons one at a time, the total number of objects in a scene can be arbitrarily large.

The Painter's Algorithm

Also called the depth sort or priority algorithm, the painter's algorithm processes polygons as if they were been painted onto the view plane in the order of their distance from the viewer. More distance polygons are painted first. Nearer polygons are painted on or over more distance polygons, partially or totally obscuring them from view. The key to implementing this concept is to find a priority ordering of the polygons in order to determine which polygons are to be painted (i.e. scan-converted) first.

Any attempt at a priority ordering based on depth sorting alone results in ambiguities that must be resolved in order to correctly assign priorities. For example, when two polygons overlap, how do we decide which one obscures the other? (See Fig. 4)

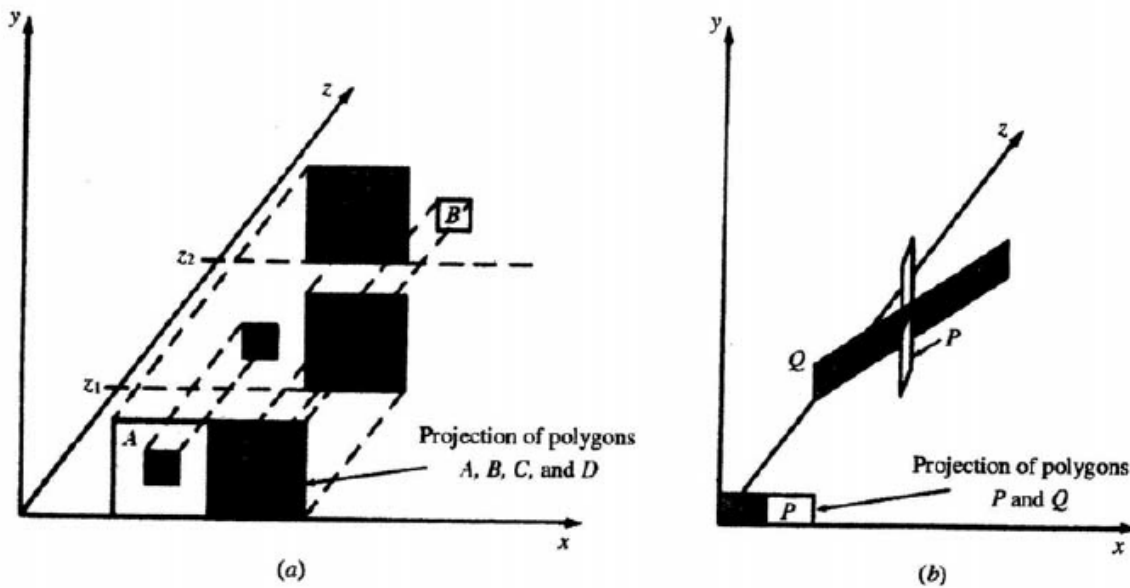


Fig. 4

Assigning Priorities

We assign priorities to polygons by determining if a given polygon P obscures other polygon. If the answer is no, then P should be painted first. Hence the key test is to determine whether polygon P does not obscure polygon Q .

The z extent of a polygon is the region between the planes $z = z_{\min}$ and $z = z_{\max}$ (Fig. 5). Here $z_{\min}$ is the smallest of the z coordinates of all the polygon's vertices, and $z_{\max}$ is the largest. Similar definitions hold for the x and y extents of a polygon. The intersection of the x , y , and z , extents is called the extent, or bounding box, of the polygon

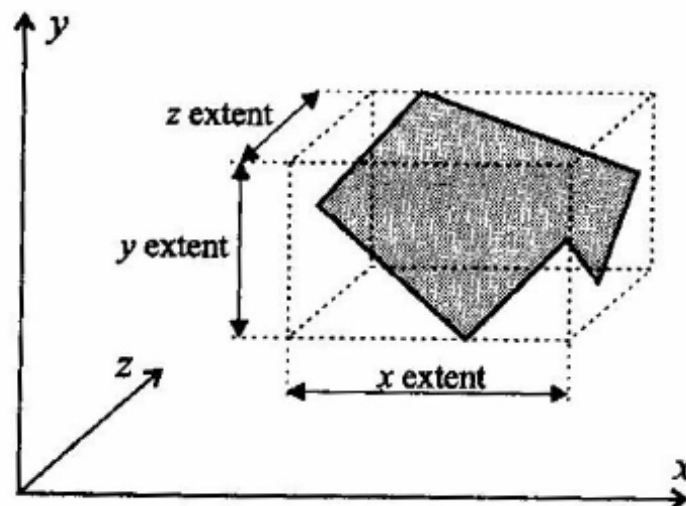


Fig. 5

Testing Whether P Obscures Q

Polygon P does not obscure polygon Q if any one of the following tests, applied in sequential order, is true.

Test 0 : the z extents of P and Q do not overlap and $z_{Q_{max}}$ of Q is smaller than $z_{P_{min}}$ of P. Refer to Fig. 6.

Test 1 : The y extents of P and Q do not overlap, Refer to Fig. 7.

Test 2 : The x extents of P and Q do not overlap.

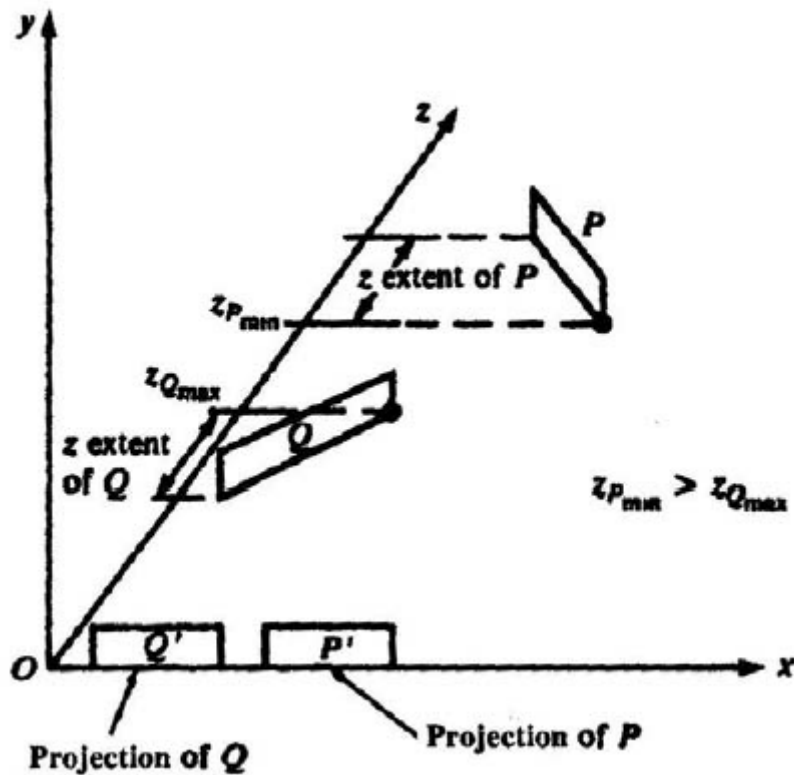


Fig. 6

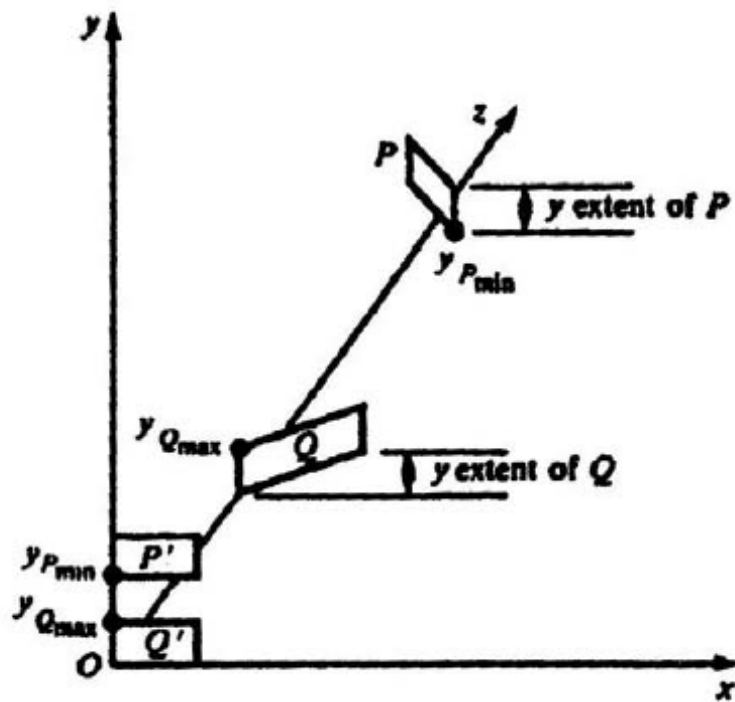


Fig. 7

Test 3 : All the vertices of P lie on that side of the plane containing Q which is farthest from the viewpoint. Refer to Fig. 8.

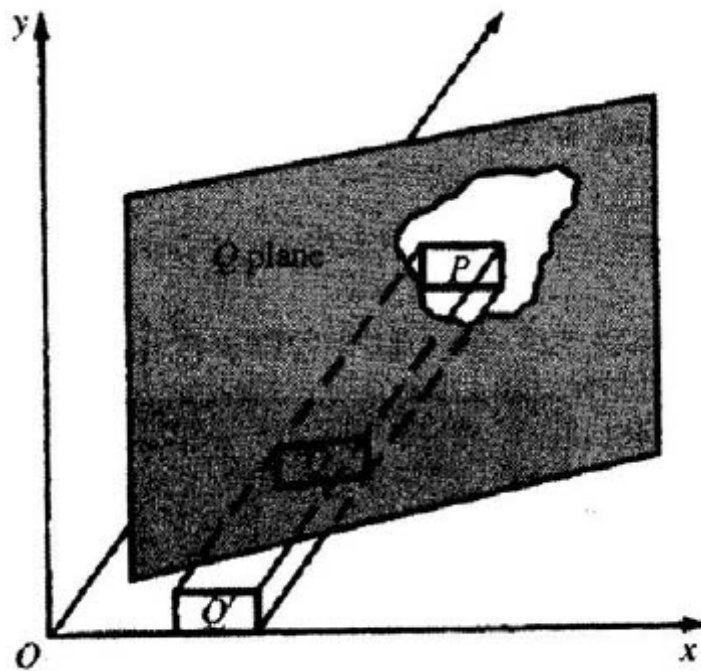


Fig. 8

Test 4: All the vertices of Q lie on that side of the plane containing P which is closest to the viewpoint. Refer to Fig. 9.

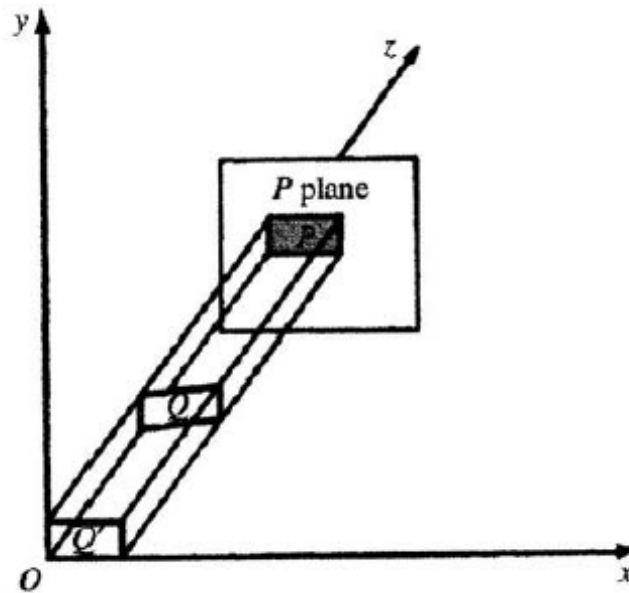


Fig. 9

Test 5 : The projections of the polygons P and Q onto the view plane do not overlap. This is checked by comparing each edge of one polygon against each edge of the other polygon to search for intersections.

The Algorithm

1. Sort all polygons into a polygon list according to $z_{\max}$ (the largest z coordinate of each polygon's vertices). Starting from the end of the list, assign priorities for each polygon P, in order, as describes in steps 2 and 3 (below).
2. Find all polygons Q (proceeding P) in the polygon list whose z extents overlap that of P (test 0).
3. For each Q, perform tests 1 through 5 until true.
 - a. If every Q passes, scan-convert polygon P.
 - b. If false for some Q, swap P and Q on the list. Tag Q as swapped. If Q has already been tagged, use the plane containing polygon P to divide polygon Q into two polygons. Q1 and Q2. Remove Q from the list and place Q1 and Q2 on the list, in sorted order.

Sometimes the polygons are subdivided into triangles before processing, thus reducing the computational effort for polygon subdivision in step 3.

Session 9: Area subdivision (Warnock), BSP tree, and Scan line.

Time (in min)	Content	Learning Aid/Methodolog y	Faculty Approach	Typical Student Activity	Skill /Competency Developed	Course Outcom e	Prog ram Out com e
10	Introduction	Presentation and Quiz	Introduces Facilitates	Participate Discuss	Knowledge Intrapersonal		
15	Area subdivision (Warnock),	Presentation Brain storming	Explain Facilitates	Listens Participate Discuss	Identify Classify Carry out		
15	BSP tree	Presentation Brain storming Quiz	Explain Facilitates	Listens Participate Discuss	Knowledge Interpret Execute Create		
15	Scan line	Presentation Brain storming Quiz	Explain Facilitates	Listens Participate Discuss	Knowledge Comprehension Implement Application		
05	Conclusion and summary	Key Words	List Facilitates Summarizes	Identifies	Knowledge Comprehension Intrapersonal		

Scan – Line Algorithm

A scan-line algorithm consists essentially of two nested loops, an x-scan loop nested within a y-scan loop.

y-Scan

For each y value, say $y = \alpha$, interested the polygons to be rendered with the scan plane $y = \alpha$. This scan plane is parallel to the xz plane, and the resulting intersections are line segments in this plane (see Fig. 10).

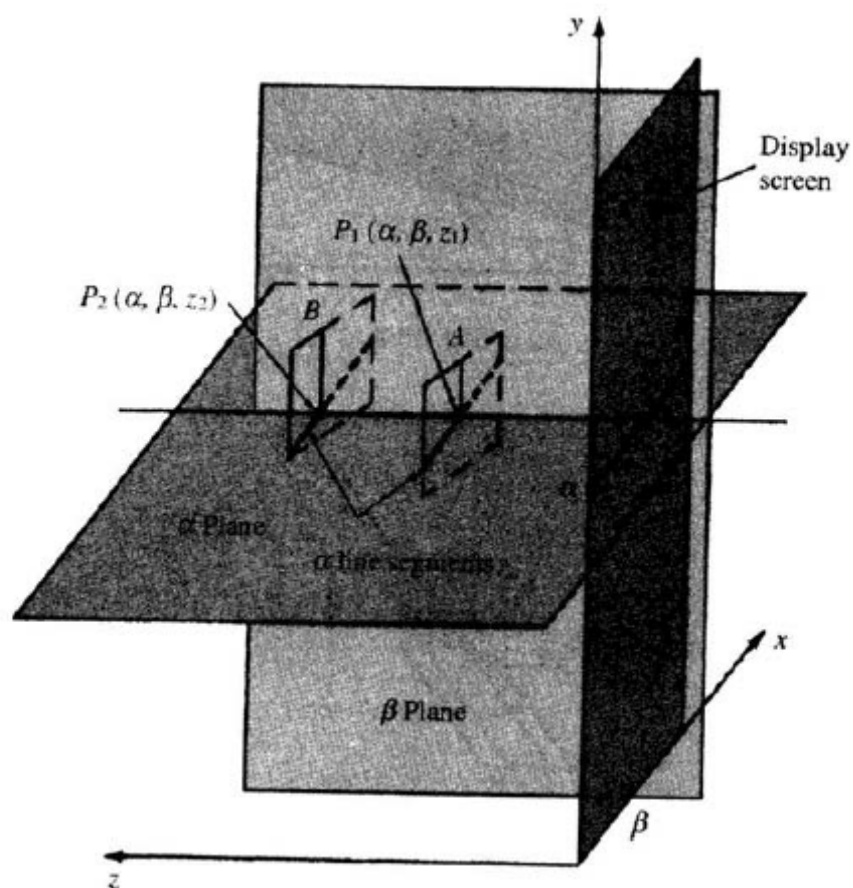


Fig. 10

x-scan

1. For each x value, say $x = f$, intersect the line segments found above with the x scan line $x = \beta$ lying on the y -scan plane. This intersection results in a set of points that lies on the x -scan line.
2. Sort these points with respect to their z coordinates. The point (x,y,z) with the smallest z value is visible, and the color of the polygon containing this point is the color set at the pixel corresponding to this point.

A Scan – line Algorithm

In the following algorithm, scan line and edge coherence are used to enhance the processing done in the y -scan as follows. Since the y -scan loop constructs a list of potentially visible line segments, instead of reconstructing this list each time the y -scan line changes (absolute calculation), we keep the list and update it according to how it has changed (incremental calculation). This processing is facilitated by the use of what is called the edge list, and its efficient construction and maintenance is at the heart of the algorithm, under “a scan-line Algorithm”).

The following data structure are created:

1. The edge list – contains all non horizontal edges (horizontal edges are automatically displayed) or the projections of the polygons in the scene. The edges are sorted by the edge's smaller y coordinate (y_{min}). Each edge entry in the edge list also contain.
 - a. The x coordinates of the end of the edge with the smaller y coordinate.
 - b. The y coordinate of the edge's other end (y_{max}).
 - c. The increment $\Delta x = 1/m$.
 - d. A pointer indicating the polygon to which the edge belongs.
2. The polygon list – for each polygon, contains
 - a. The equation of the plane within which the polygon lies – used for depth determination, i.e., to find the z value at pixel (x, y) .
 - b. An IN/OUT flag, initialized to OUT (this flag is set depending on whether a given scan line is in or out of the polygon).
 - c. Color information for the polygon.

The algorithm proceeds as follows.

I. Initialization

- a. Initialize each screen pixel to a background color.
- b. Set y to the smallest y_{min} value in the edge list.

Repeat steps II and III (below) until further processing can be performed.

II. y-scan loop. Activate edges whose y min is equal to y. Sort active edges in order of increasing x.

III. x – scan loop. Process, from left to right, each active edge as follows:

a. Invert the IN / OUT flag of the polygon in the polygon list which contains the edge. Count the number of active polygon whose IN/OUT flag is set to IN. If this number is 1, only one polygon is visible. All pixel values from this edge and up to the next edge are set to the color of then polygon. If this number is greater than 1, determine the visible polygon by the smallest z value of each polygon at the pixel under consideration.

These z values are found from the equation of the plane containing the polygon, unless the polygon becomes obscured by another before the next edge is reached, in which case we set the remaining pixels to the color of the obscuring polygon. If this number is 0, pixels from this edge and up to the next one are left unchanged.

b. When the last active edge is processed, we then proceed as follows :

1. Remove those edges for which the value of y max equals the present scan-line value y. If no edges remain, the algorithm has finished.
2. For each remaining active edge, in order, replace x by $x + 1/m$. This is the edge intersection with the next scan line $y + 1$.
3. Increment y to $y + 1$, the next scan line, and repeat step II.

Subdivision Algorithm

The subdivision algorithm is a recursive procedure based on a two – step strategy that first decides which projected polygons overlap a given area A on the screen and are therefore potentially visible in that area. Second, in each area these polygons are further tested to determine which ones will be visible within this area and should therefore be displayed. If a visibility decision cannot be made, this screen area, usually a rectangular window, is further subdivided either until a visibility decision can be made, or until the screen area is a single pixel.

Starting with the full screen as the initial area, the algorithm divides an area at each stage into four smaller areas, thereby generating a quad tree (see Fig. 11).

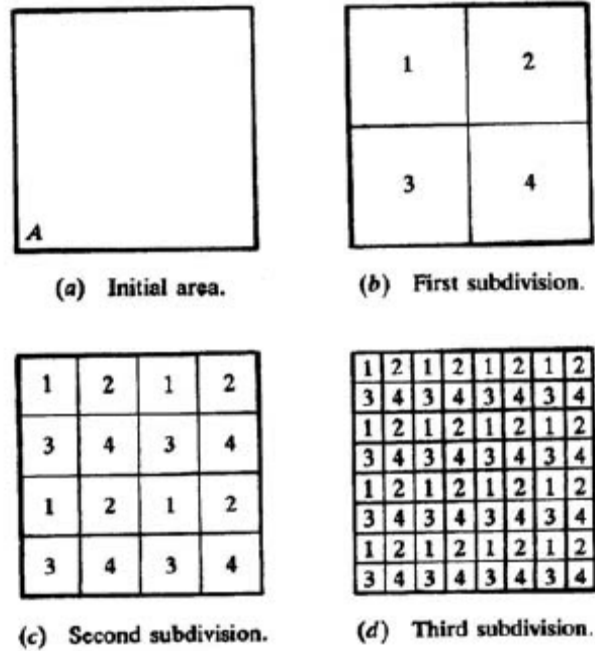


Fig. 11

The processing exploits area coherence by classifying polygon P with respect to a given screen A into the following categories ; (1) surrounding polygon – polygon that completely contains the area [Fig.12(a)], (2) intersecting polygon – polygon that intersects the area [Fig. 12(b)], (3) contained polygon – polygon that is completely contained within the area [Fig. 12(c)], and (4) disjoint polygon – polygon that is disjoint from the area [Fig. 12(d)].

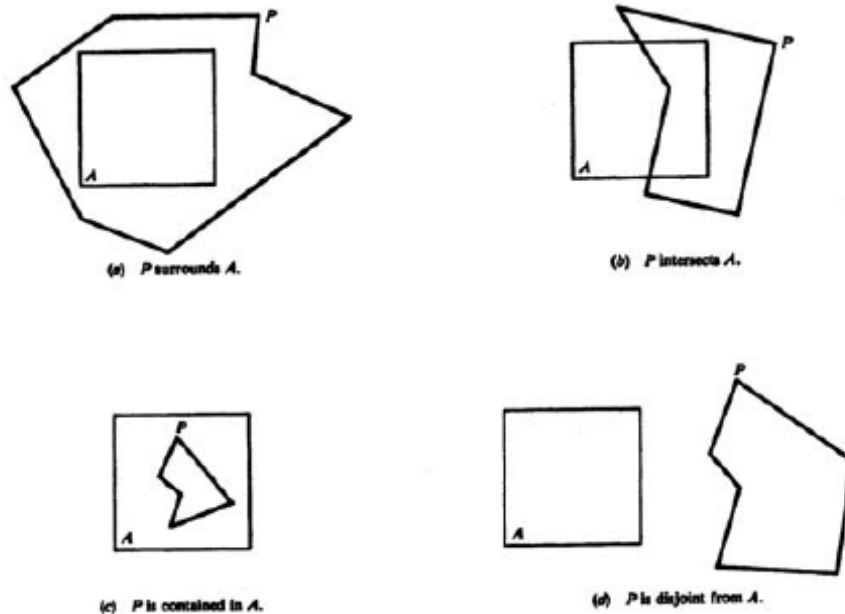


Fig. 12

The Classification of the polygons within a picture is the main computational expense of the algorithm and is analogous to the clipping algorithms discussed in. With the use of one of these clipping algorithms, a polygon in category 2 (intersecting polygon) can be clipped into a contained polygon and a disjoint polygon (see Fig. 13). Therefore, we could proceed as if category 2 were eliminated.

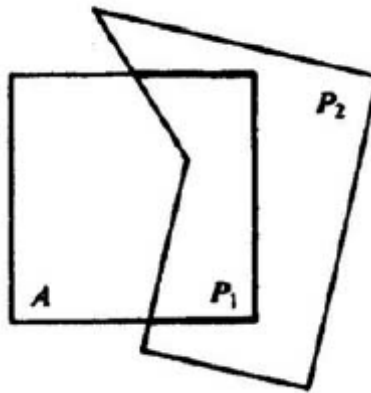


Fig. 13

For a given screen area, we keep a potentially visible polygons list (PVPL), those in categories 1,2, and 3 (Disjoint polygons are clearly not visible). Also, note that on subdivision of a screen area, surrounding and disjoint polygons remain surrounding and disjoint polygons of the newly formed areas. Therefore, only contained and intersecting polygons need to be reclassified.

Removing Polygons Hidden by a Surrounding Polygon

The key to efficient visibility computation lies in the fact that a polygon is not visible if it is in back of a surrounding polygon. Therefore, it can be removed from the PVPL. To facilitate processing, this list is sorted by z_{min} , the smallest z coordinate of the polygon within this area.

In addition for each surrounding polygon S , we also record its largest z coordinate $z_{s\ max}$.

If, for a polygon P on the list $z_{p\ min} > z_{s\ max}$ (for a surrounding polygon S), then P is hidden by S and thus is not visible. In addition, all other polygons after P on the list will also be hidden by S , so we can remove these polygons from the PVPL.

Subdivision Algorithm

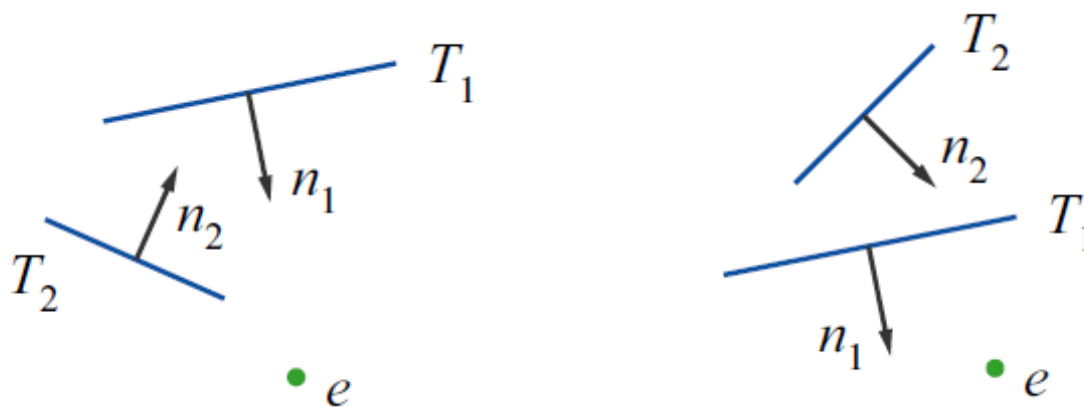
1. Initialize the area to be whole screen.

2. Create a PVPL with respect to an area sorted on z min (the smallest z coordinate of the polygon within the area). Place the polygons in their appropriate categories. Remove polygons hidden by a surrounding polygon and remove disjoint polygons.
3. Perform the visibility decision tests:
 - a. If the list is empty, set all pixels to the background color.
 - b. If there is exactly one polygon in the list and it is classified as intersecting (category 2) or contained (category 3), color (scan – convert) the polygon, and color the remaining area to the background color.
 - c. If there is exactly one polygon on the list and it is surrounding one, color the area the color of the surrounding polygon.
 - d. If the area is the pixel (x, y) and neither a, b, nor c applies, compute the z coordinate $z(x, y)$ at pixel (x, y) of all polygons on the PVPL. The pixel is then set to the color of the polygon with the smallest z coordinate.
4. If none of the above cases has occurred, subdivide the screen area into fourths. For each area, go to step 2.

Binary Space Partitioning Trees

The idea of **binary space partitioning trees** (BSP trees) is to extend the painter's algorithm to make back-to-front ordering of polygons fast for any eye location and to divide polygons to avoid overlaps.

Imagine two patches, T_1 and T_2 , with outward-facing normal $\vec{n}_1$ and $\vec{n}_2$.



If the eye point, $\vec{e}$, and T_2 are on the same side of T_1 , then we draw T_1 before T_2 . Otherwise, T_2 should be drawn before T_1 .

We know if two points are on the same side of a plane containing T_1 by using the implicit equation for T_1 ,

$$fl(\vec{x}) = (\vec{x} - \vec{p1}) \cdot \vec{n}.$$

If $\vec{x}$ is on the plane, $fl(\vec{x}) = 0$. Otherwise, if $fl(\vec{x}) > 0$, $\vec{x}$ is on the “outside” of T1, and if

$fl(\vec{x}) < 0$, $\vec{x}$ is “inside.”

Before any rendering can occur, the scene geometry must be processed to build a BSP tree to represent the relative positions of all the facets with respect to their inside/outside half-planes. The same BSP tree can be used for any eye position, so the tree only has to be constructed once if everything other than the eye is static. For a single scene, there are many different BSP trees that can be used to represent it— it’s best to try to construct balanced trees.

The tree traversal algorithm to draw a tree with root F is as follows:

```

if eye is in the outside half-space of F
    draw faces on the inside subtree of F
    draw F
    draw faces on the outside subtree of F
else
    draw faces on the outside subtree of F
    draw F (if backfaces are drawn)
    draw
end

```

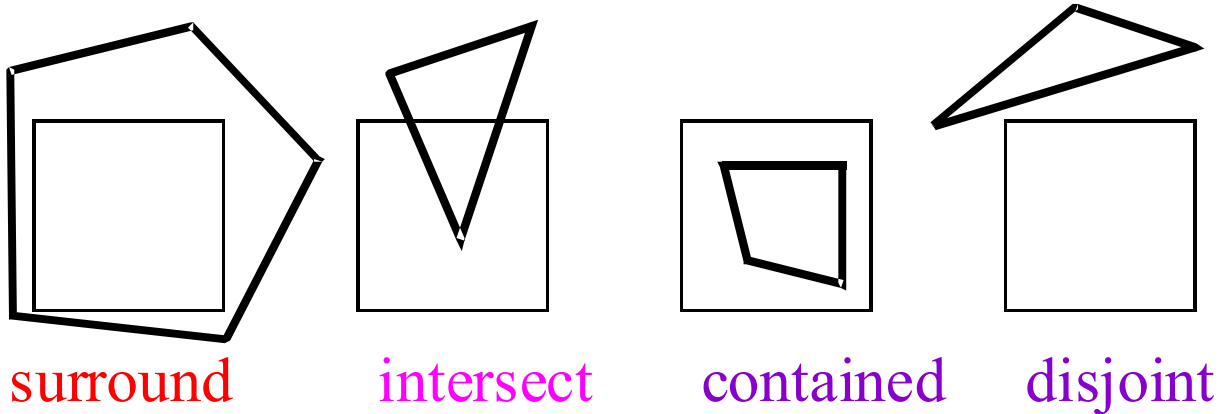
Warnock’s Algorithm

- It uses Divide and Conquer approach to remove hidden surfaces.
- It starts with the total view.
- Tests are applied to decide whether scene can be displayed or scene is complex and we should subdivide the total area into smaller rectangles.
- These tests are then applied to each smaller area, subdividing these if the tests indicate that visibility of single surface is still uncertain.
- This subdivision process is continued until the subdivided area can be easily analyzed or they are reduced to the size of single pixel.

Four Tests : Warnock’s Algorithm

Tests : To determine the visibility of single surface(polygon) within a specified area
 How tests are carried out ? : by giving the relationship of a display area and a polygon.

1. **Surrounding Polygon:** Polygon completely encloses the area
2. **Intersecting polygon:** polygon that is partly inside and partly outside the area.
3. **Contained Polygon:** polygon is completely inside the area.
4. **Disjoint Polygon:** polygon is completely outside the area.



Warnock's Algorithm

- Starting from the entire display area, we check the following four cases. If none holds, we subdivide the area, otherwise, we stop and perform the action associated with the case
 - All polygons are disjoint wrt the area -> draw the area with background color
 - Only 1 intersecting or contained polygon -> draw the area with background, and then draw the contained portion of the polygon
 - There is a single surrounding polygon -> draw the entire area in the polygon's color
 - There are more than one intersecting, contained, or surrounding polygons, but there is a front surrounding polygon -> draw the entire area in the polygon's color
- The recursion stops when you are at the pixel level

At Single Pixel Level

- When the recursion stop and none of the four cases hold, we need to perform depth sort and draw the polygon with the closest Z value
- The algorithm is done at the object space level, except scan conversion and clipping are done at the image space level

